

[Assignment VIII]

Problem 2: Consider the optimization problem

$$\underset{\theta \in \Theta}{\text{maximize}} \quad f(\theta).$$

Informally assume f is an intractable function, i.e., evaluating $f(\theta)$ is difficult. However, assume there exists a decomposition

$$f(\theta) = g(\theta, \phi) + h(\theta, \phi) \quad \forall \phi \in \Phi,$$

where g and h are tractable, i.e., evaluating $g(\theta, \phi)$ and $h(\theta, \phi)$ are easy, $h(\theta, \phi) \geq 0$ for all $\theta \in \Theta$ and $\phi \in \Phi$, and for any $\theta \in \Theta$ there exists a $\phi \in \Phi$ such that $h(\theta, \phi) = 0$, i.e., for any $\theta \in \Theta$,

$$\min_{\phi \in \Phi} h(\theta, \phi) = 0$$

and the minimum is attained. Now we consider the following problem with the tractable objective function.

$$\underset{\theta \in \Theta, \phi \in \Phi}{\text{maximize}} \quad g(\theta, \phi).$$

Show that the two optimization problems are equivalent in the sense that

$$\operatorname{argmax} f = \{\theta \mid (\theta, \phi) \in \operatorname{argmax} g\}$$

Hint. Use the fact that

$$\sup_{\theta, \phi} g(\theta, \phi) = \sup_{\theta} \left(\sup_{\phi} g(\theta, \phi) \right).$$

$$\begin{aligned} \tilde{\theta} \in \operatorname{argmax} f &\Rightarrow \exists \tilde{\phi} \in \Phi \quad h(\tilde{\theta}, \tilde{\phi}) = 0, \quad g(\tilde{\theta}, \tilde{\phi}) = f(\tilde{\theta}) - h(\tilde{\theta}, \tilde{\phi}) \leq f(\tilde{\theta}) \leq f(\tilde{\theta}) = g(\tilde{\theta}, \tilde{\phi}) + h(\tilde{\theta}, \tilde{\phi}) = g(\tilde{\theta}, \tilde{\phi}) \\ &\Rightarrow (\tilde{\theta}, \tilde{\phi}) \in \operatorname{argmax} g \end{aligned}$$

$$\begin{aligned} (\tilde{\theta}, \tilde{\phi}) \in \operatorname{argmax} g &\Rightarrow \exists \hat{\phi} \in \Phi \quad h(\tilde{\theta}, \hat{\phi}) = 0, \quad \underbrace{g(\tilde{\theta}, \tilde{\phi})}_{\operatorname{argmax}} = f(\tilde{\theta}) - h(\tilde{\theta}, \tilde{\phi}) = g(\tilde{\theta}, \hat{\phi}) + h(\tilde{\theta}, \hat{\phi}) - h(\tilde{\theta}, \tilde{\phi}) \leq g(\tilde{\theta}, \hat{\phi}) \\ &\Rightarrow h(\tilde{\theta}, \tilde{\phi}) = 0, \quad f(\tilde{\theta}) = g(\tilde{\theta}, \tilde{\phi}) \\ &\Rightarrow \forall \theta \exists \phi \in \Phi \quad h(\theta, \phi) = 0, \quad f(\theta) = g(\theta, \phi) + h(\theta, \phi) = g(\theta, \phi) \leq g(\tilde{\theta}, \tilde{\phi}) = f(\tilde{\theta}) \\ &\Rightarrow \tilde{\theta} \in \operatorname{argmax} f \end{aligned}$$

Therefore those two optimisation problems are equivalent

Problem 3: Inverse permutation. Let S_n denote the group of length n permutations. Note that the map $i \mapsto \sigma(i)$ is a bijection. Define $\sigma^{-1} \in S_n$ as the permutation representing the inverse of this map, i.e, $\sigma^{-1}(\sigma(i)) = i$ for $i = 1, \dots, n$. Describe an algorithm for computing σ^{-1} given σ .

Clarification. In this class, defined σ as a list of length n containing $\{1, \dots, n\}$ exactly once. The output of the algorithm, σ^{-1} , should also be provided as a list.

$$\sigma : [\sigma(1), \sigma(2), \dots, \sigma(n)]^t$$

$$\sigma^{-1} : [(\sigma^{-1}(1), \sigma^{-1}(2), \dots, \sigma^{-1}(n))]^t$$

$$\downarrow [1, \sigma^{-1}(\sigma(1)), 2, \sigma^{-1}(\sigma(2)), \dots, n, \sigma^{-1}(\sigma(n))]^t$$

$$\downarrow [\sigma(1), \dots, \sigma(n)]^t \text{ or } [1, 2, \dots, n]^t \text{ or } \sigma$$

Problem 4: Permutation matrix. Given a permutation $\sigma \in S_n$, the *permutation matrix* of σ is defined as

$$P_\sigma = \begin{bmatrix} e_{\sigma(1)}^T \\ e_{\sigma(2)}^T \\ \vdots \\ e_{\sigma(n)}^T \end{bmatrix} \in \mathbb{R}^{n \times n},$$

where $e_1, \dots, e_n \in \mathbb{R}^n$ are the standard unit vectors. Show

(a) $P_\sigma^T = P_\sigma^{-1} = P_{\sigma^{-1}}$ and

(b) $|\det P_\sigma| = 1$.

Hint. If the rows of $U \in \mathbb{R}^{n \times n}$ are orthonormal, we say U is an orthogonal matrix. Orthogonal matrices satisfy $UU^T = U^TU = I$.

(a) $P_\sigma^T = [e_{\sigma(1)} \dots e_{\sigma(n)}] \Rightarrow [P_\sigma^T P_\sigma]_{ij} = \langle e_{\sigma(i)}, e_{\sigma(j)} \rangle = \delta_{ij}$, $P_\sigma^T P_\sigma = I_n$, $P_\sigma^T = P_\sigma^{-1}$

$P_\sigma^{-1} P_\sigma = I$ $\Leftrightarrow P_\sigma^T P_\sigma^{-T} = I = [e_{\sigma(1)} \dots e_{\sigma(n)}][e_{\sigma^{-1}(1)} \dots e_{\sigma^{-1}(n)}]$?

(L) $[e_{\sigma(1)} \dots e_{\sigma(n)}][e_{\sigma^{-1}(1)} \dots e_{\sigma^{-1}(n)}] e_i$

$= [e_{\sigma(1)} \dots e_{\sigma(n)}] e_{\sigma^{-1}(i)} = e_{\sigma\sigma^{-1}(i)} = e_i \quad \forall i \in \{1, 2, \dots, n\}$

$\Rightarrow$ Linear Extension Theorem of $[e_{\sigma(1)} \dots e_{\sigma(n)}][e_{\sigma^{-1}(1)} \dots e_{\sigma^{-1}(n)}] = \text{id} = I_n$

$\Rightarrow P_\sigma^{-1} = P_\sigma^{-T} = P_\sigma^T$

(b) $P_\sigma^T P_\sigma = I \Rightarrow \det(P_\sigma^T P_\sigma) = \det(P_\sigma)^2 = \det(I_n) = 1 \Rightarrow |\det P_\sigma| = 1$

(In fact, $\det P_\sigma = \text{sgn } \sigma$. It can be proved using transposition and elementary matrix theory, because every permutation can be represented by compositions of transpositions.)

Problem 5: Affine coupling layer with permutations. Consider the affine coupling layer defined as follows. Let $\Omega \subseteq \{1, \dots, n\}$ and $0 < |\Omega| < n$. Define $\Omega^c = \{1, \dots, n\} \setminus \Omega$. For $x \in \mathbb{R}^n$, define

$$x_\Omega \in \mathbb{R}^{|\Omega|}, \quad x_{\Omega^c} \in \mathbb{R}^{n-|\Omega|}$$

to be the sub-vectors of x with the indices within Ω and Ω^c selected. Define z_Ω and z_{Ω^c} analogously for $z \in \mathbb{R}^n$. The affine coupling layer as

$$z_\Omega = x_\Omega$$

$$z_{\Omega^c} = e^{s_\theta(x_\Omega)} \odot x_{\Omega^c} + t_\theta(x_\Omega),$$

where $s_\theta: \mathbb{R}^{|\Omega|} \rightarrow \mathbb{R}^{n-|\Omega|}$ and $t_\theta: \mathbb{R}^{|\Omega|} \rightarrow \mathbb{R}^{n-|\Omega|}$. Show that

$$\log \left| \frac{\partial z}{\partial x} \right| = \mathbf{1}_{(n-|\Omega|)}^T s_\theta(x_\Omega).$$

Clarification. We are not assuming $|\Omega| = n/2$.

Hint. Find a permutation σ such that

$$\frac{\partial z}{\partial x} = P_{\sigma^{-1}} \begin{bmatrix} I & 0 \\ * & \text{diag}(e^{s_\theta(x_\Omega)}) \end{bmatrix} P_\sigma.$$

Let $\sigma \in S_n$ be an arbitrary permutation such that σ satisfies $\sigma(1), \dots, \sigma(|\Omega|) \in \Omega$

Reorder x, z as x_σ, z_σ .

$$\frac{\partial z}{\partial x} = \frac{\partial z_\sigma}{\partial x_\sigma} \frac{\partial x_\sigma}{\partial x} = P_\sigma \begin{bmatrix} I & 0 \\ * & \text{diag}(e^{s_\theta(x_\Omega)}) \end{bmatrix} P_\sigma$$

($\because \frac{\partial z_i}{\partial x_j} = f_{ij} \quad (1 \leq i, j \leq |\Omega|)$) $\rightarrow$ const.

$$\frac{\partial z_i}{\partial x_j} = \frac{\partial}{\partial x_j} (e^{\text{const.}} x_i + t_\theta(x_\Omega)_i)$$

$$= f_{ij} e^{s_\theta(x_\Omega)_i}$$

$$\left| \frac{\partial z}{\partial x} \right| = \cancel{\det P_\sigma^{-1}} \det I \det(\text{diag}(e^{s_\theta(x_\Omega)})) \cancel{\det P_\sigma}$$

$$\log \left(\frac{\partial z}{\partial x} \right) = \sum_{i=1}^{n-|\Omega|} s_\theta(x_\Omega)_{\sigma(i)} = \mathbf{1}_{(n-|\Omega|)}^T s_\theta(x_\Omega)_\sigma$$

(ex) $\sigma = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 3 & 2 & 4 & 1 \end{pmatrix}$

$$\frac{\partial x_\sigma}{\partial x} = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{pmatrix} = P_\sigma$$

Problem 8: Log-derivative trick for VAE. Let $Z \in \mathbb{R}^k$ be a random variable. Let $q_\phi(z)$ be a probability density function for all $\phi \in \mathbb{R}^p$. Assume $q_\phi(z)$ is differentiable in ϕ for all fixed $z \in \mathbb{R}^k$. Let $h: \mathbb{R}^k \rightarrow \mathbb{R}$ satisfying $h(z) > 0$ for all $z \in \mathbb{R}^k$. Assume that the order of integration and differentiation can be swapped. Show

$$\nabla_\phi \mathbb{E}_{Z \sim q_\phi(z)} \left[\log \left(\frac{h(Z)}{q_\phi(Z)} \right) \right] = \mathbb{E}_{Z \sim q_\phi(z)} \left[(\nabla_\phi \log q_\phi(Z)) \log \left(\frac{h(Z)}{q_\phi(Z)} \right) \right].$$

Hint. Since $q_\phi(z)$ is a probability density function,

$$\int \nabla_\phi q_\phi(z) dz = \nabla_\phi \int q_\phi(z) dz = \nabla_\phi 1 = 0.$$

$$\nabla_\phi \mathbb{E}_{Z \sim q_\phi(z)} \left[\log \left(\frac{h(Z)}{q_\phi(Z)} \right) \right] = \nabla_\phi \int \log \frac{h(z)}{q_\phi(z)} q_\phi(z) dz = \nabla_\phi \left(\int \log h(z) q_\phi(z) dz - \int \log q_\phi(z) q_\phi(z) dz \right)$$

$$= \int \log h(z) \nabla_\phi q_\phi(z) dz - \left(\int \frac{\nabla_\phi q_\phi(z)}{q_\phi(z)} q_\phi(z) dz + \int (\nabla_\phi q_\phi(z)) \log q_\phi(z) dz \right)$$

$$= \int \log h(z) \nabla_\phi q_\phi(z) dz - \left(\cancel{\int q_\phi(z) dz}^{=1} + \int (\nabla_\phi q_\phi(z)) \log q_\phi(z) dz \right)$$

$$= \int (\nabla_\phi q_\phi(z)) \log \frac{h(z)}{q_\phi(z)} dz = \int \frac{\nabla_\phi q_\phi(z)}{q_\phi(z)} \log \frac{h(z)}{q_\phi(z)} q_\phi(z) dz$$

$$= \int (\nabla_\phi \log q_\phi(z)) \log \frac{h(z)}{q_\phi(z)} q_\phi(z) dz$$

$$= \mathbb{E}_{Z \sim q_\phi(z)} \left[(\nabla_\phi \log q_\phi(Z)) \log \frac{h(Z)}{q_\phi(Z)} \right] \quad \square$$

MATHDNN_HW8

November 30, 2021

1 Problem 1

```
[ ]: import torch
import torch.utils.data as data
import torch.nn as nn
from torch.distributions.normal import Normal
from torch.distributions.uniform import Uniform
import numpy as np
import matplotlib.pyplot as plt
```

```
[ ]: device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

import warnings
warnings.filterwarnings("ignore")
```

```
[ ]: epochs = 1000
learning_rate = 5e-3
batch_size = 128
n_components=5 # the number of kernel
target_distribution = Normal(0.0, 1.0)
```

```
[ ]: #####
# STEP 1: Implement 1-d Flow model #
# Model is mixture of Gaussian CDFs #
#####

class Flow1d(nn.Module):
    def __init__(self, n_components):
        super(Flow1d, self).__init__()
        self.mus = nn.Parameter(torch.randn(n_components), requires_grad=True)
        self.log_sigmas = nn.Parameter(torch.zeros(n_components),
        ↳requires_grad=True)
        self.weight_logits = nn.Parameter(torch.ones(n_components),
        ↳requires_grad=True)

    def forward(self, x):
        x = x.view(-1,1)
```

```

    # implement this
    weights = self.weight_logits.exp().view(1,-1)
    distribution = Normal(self.mus, self.log_sigmas.exp())
    z = ((distribution.cdf(x)-0.5) * weights).sum(dim=1)
    dz_by_dx = (distribution.log_prob(x).exp() * weights).sum(dim=1)
    return z, dz_by_dx

```

```

[ ]: #####
# STEP 2: Create Dataset and Create Dataloader #
#####

def mixture_of_gaussians(num, mu_var=(-1,0.25, 0.2,0.25, 1.5,0.25)):
    n = num // 3
    m1,s1,m2,s2,m3,s3 = mu_var
    gaussian1 = np.random.normal(loc=m1, scale=s1, size=(n,))
    gaussian2 = np.random.normal(loc=m2, scale=s2, size=(n,))
    gaussian3 = np.random.normal(loc=m3, scale=s3, size=(num-n,))
    return np.concatenate([gaussian1, gaussian2, gaussian3])

class MyDataset(data.Dataset):
    def __init__(self, array):
        super().__init__()
        self.array = array

    def __len__(self):
        return len(self.array)

    def __getitem__(self, index):
        return self.array[index]

```

```

[ ]: #####
# STEP 3: Define Loss Function #
#####

def loss_function(target_distribution, z, dz_by_dx):
    # log(p_Z(z)) = target_distribution.log_prob(z)
    # log(dz/dx) = dz_by_dx.log() (flow is defined so that dz/dx>0)
    log_likelihood = target_distribution.log_prob(z) + dz_by_dx.log()
    return -log_likelihood.mean() #flip sign, and sum of data X_1,...X_N

```

```

[ ]: #####
# STEP 4: Train the model #
#####

# create dataloader
n_train, n_test = 5000, 1000
train_data = mixture_of_gaussians(n_train)
test_data = mixture_of_gaussians(n_test)

```

```

train_loader = data.DataLoader(MyDataset(train_data), batch_size=batch_size,
    ↪shuffle=True)
test_loader = data.DataLoader(MyDataset(test_data), batch_size=batch_size,
    ↪shuffle=True)

# create model
flow = Flow1d(n_components)
optimizer = torch.optim.Adam(flow.parameters(), lr=learning_rate)

train_losses, test_losses = [], []

for epoch in range(epochs):
    # train
    # flow.train()
    mean_loss = 0
    for i, x in enumerate(train_loader):
        z, dz_by_dx = flow(x)
        loss = loss_function(target_distribution, z, dz_by_dx)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        mean_loss += loss.item()
    train_losses.append(mean_loss/(i+1))

    # test
    flow.eval()
    mean_loss = 0
    for i, x in enumerate(test_loader):
        z, dz_by_dx = flow(x)
        loss = loss_function(target_distribution, z, dz_by_dx)

        mean_loss += loss.item()
    test_losses.append(mean_loss/(i+1))

```

```

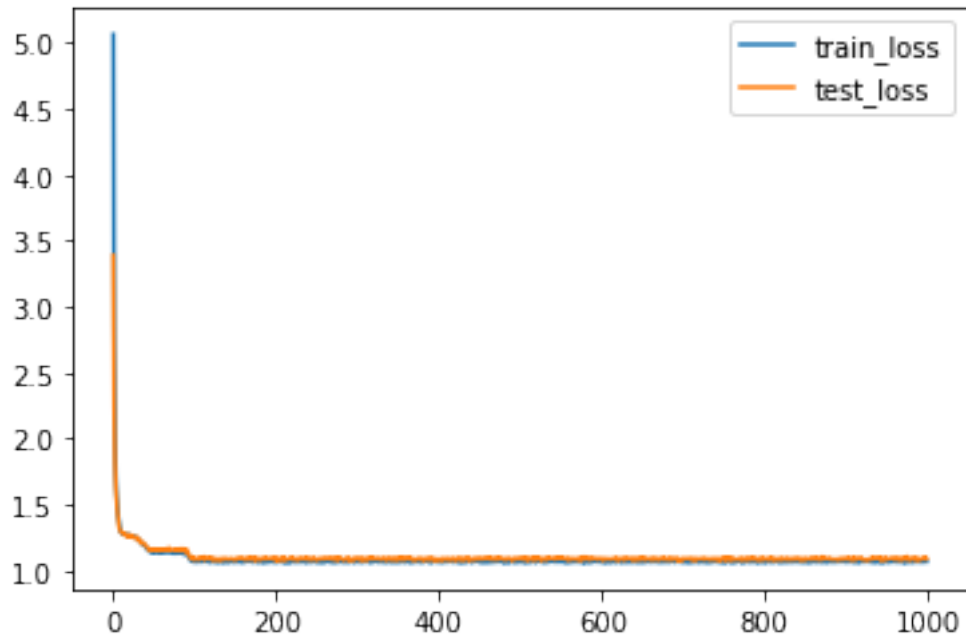
[ ]: plt.plot(train_losses, label='train_loss')
plt.plot(test_losses, label='test_loss')
plt.legend()

```

```

[ ]: <matplotlib.legend.Legend at 0x7f3c5961fe90>

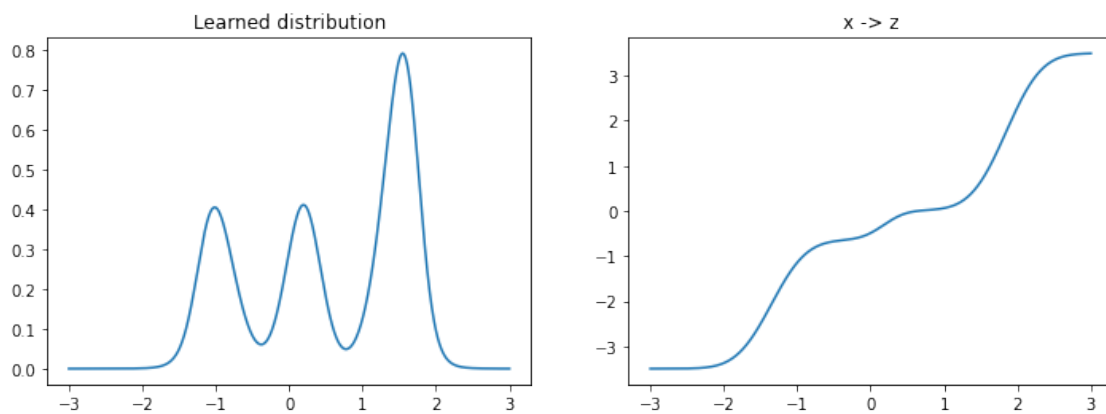
```

```
[ ]: x = np.linspace(-3,3,1000)
with torch.no_grad():
    z, dz_by_dx = flow(torch.FloatTensor(x))
    px = (target_distribution.log_prob(z) + dz_by_dx.log()).exp().cpu().numpy()

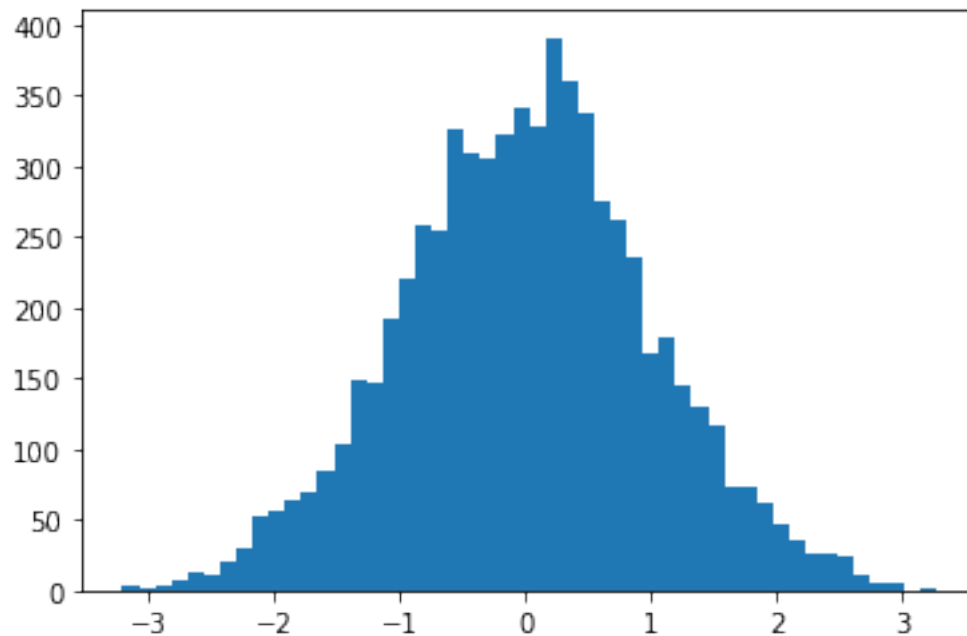
_, axes = plt.subplots(1,2, figsize=(12,4))
_ = axes[0].plot(x,px)
_ = axes[0].set_title('Learned distribution')

_ = axes[1].plot(x,z)
_ = axes[1].set_title('x -> z')
```



```
[ ]: with torch.no_grad():
      z, _ = flow(torch.FloatTensor(train_loader.dataset.array))

      _ = plt.hist(np.array(z), bins=50)
```



2 Problem 6

```
[1]: N = 3000
      K = 600
      p = 18/37
      q = 0.55
```

```
[2]: import torch
```

```
[3]: arr = torch.ones(N,K)
      arr = torch.bernoulli(arr*q) # 600 * 3000
```

```
[16]: def phi(X):
        pos = 100
        for i in range(len(X)):
            pos += 2*X[i]-1
            if pos >= 200:
                return 1
            elif pos <= 0:
                return 0
```

```

    return 0

def f(X):
    return torch.pow(p,X.sum().double())*torch.pow(1-p,(K-X.sum()).double())

def g(X):
    return torch.pow(q,X.sum().double())*torch.pow(1-q,(K-X.sum()).double())

```

```

[17]: print(f(arr[1,:]))
      print(g(arr[1,:]))
      print(phi(arr[1,:]))

```

```

tensor(4.4958e-182, dtype=torch.float64)
tensor(2.6645e-180, dtype=torch.float64)
0

```

```

[18]: s = torch.mean(torch.tensor([f(arr[i,:]) / g(arr[i, :]) * phi(arr[i, :]) for i_
      ↪in range(N)]))

```

```

[21]: print(f"probability : {s.tolist()}")

```

```

probability : 2.2059165179411235e-06

```

3 Problem 7

(a) the log-derivative trick

```

[ ]: import torch
      import math
      import numpy as np

      mu = 0
      tau = 0
      lr = 1e-2
      B = 25
      iterations = 2000 # not epochs, iterations
      history1 = torch.zeros((iterations+1),2)

      history1[0][1] = 1

      for itr in range(iterations):
          X = torch.normal(mu,np.exp(tau),size=(B,))
          g = torch.sum(X*torch.sin(X)*(X-mu)* (np.exp(-2*tau)))/B + mu - 1
          h = torch.sum(X*torch.sin(X)*((X-mu)**2 * np.exp(-2*tau) -1))/B + np.
      ↪exp(tau) -1
          mu -= lr*g
          tau -= lr*h
          # save history

```

```

    history1[itr+1][0] = mu
    history1[itr+1][1] = np.exp(tau)

print(f"mu : {mu.tolist()}, sigma : {np.exp(tau.tolist())}")

```

mu : 0.4355337619781494, sigma : 0.5916807390900852

(b) the reparameterisation trick

```

[ ]: import torch
import math
import numpy as np

mu = 0
tau = 0
lr = 1e-2
B = 25
iterations = 2000 # not epochs, iterations
history2 = torch.zeros((iterations+1, 2))

history2[0][1] = 1

for itr in range(iterations):
    Y = torch.normal(0,1,size=(B,))
    g = torch.sum((np.exp(tau)*Y+mu)*torch.cos(np.exp(tau)*Y+mu)+torch.sin(np.
    ↪exp(tau)*Y+mu))/B + mu - 1
    h = torch.sum(((np.exp(tau)*Y+mu)*torch.cos(np.exp(tau)*Y+mu)+torch.sin(np.
    ↪exp(tau)*Y+mu))*np.exp(tau)*Y)/B + np.exp(tau) - 1
    mu -= lr*g
    tau -= lr*h
    # save history
    history2[itr+1][0] = mu
    history2[itr+1][1] = np.exp(tau)

print(f"mu : {mu.tolist()}, sigma : {np.exp(tau.tolist())}")

```

mu : 0.42898011207580566, sigma : 0.607804746827791

compare

```

[ ]: import numpy as np
import matplotlib.pyplot as plt

%matplotlib inline

x1 = np.array(history1[:, 0])
y1 = np.array(history1[:, 1])

x2 = np.array(history2[:, 0])

```

```

y2 = np.array(history2[:, 1])

fig, (ax1, ax2) = plt.subplots(1, 2)

ax1.scatter(0,1, s=100, c='red')
ax1.plot(x1, y1, linestyle='solid',color='blue')
ax1.set_title('log derivative trick')

ax2.scatter(0,1, s=100, c='red')
ax2.plot(x2, y2, linestyle='solid',color='blue')
ax2.set_title('reparametrisation trick')

plt.tight_layout()
#plt.savefig('derivative_trick.png')
plt.show()

```

